

the standard techniques of data and functions. Later, I'll rewrite these programs using OOP techniques. The purpose of this chapter is to demonstrate some key problems inherent in procedural programming. With that understanding, the chapters that follow will explain how OOP solves those problems.

Higher or Lower Card Game

My first example is a simple card game called Higher or Lower. In this game, eight cards are randomly chosen from a deck. The first card is shown face up. The game asks the player to predict whether the next card in the selection will have a higher or lower value than the currently showing card. For example, say the card that's shown is a 3. The player chooses "higher," and the next card is shown. If that card has a higher value, the player is correct. In this example, if the player had chosen "lower," they would have been incorrect.

If the player guesses correctly, they get 20 points. If they choose incorrectly, they lose 15 points. If the next card to be turned over has the same value as the previous card, the player is incorrect.

Representing the Data

The program needs to represent a deck of 52 cards, which I'll build as a list. Each of the 52 elements in the list will be a dictionary (a set of key/value pairs). To represent any card, each dictionary will contain three key/value pairs: 'rank', 'suit', and 'value'. The rank is the name of the card (Ace, 2, 3, ... 10, Jack, Queen, King), but the value is an integer used for comparing cards (1, 2, 3, ... 10, 11, 12, 13). For example, the Jack of Clubs would be represented as the following dictionary:

```
{ 'rank': 'Jack', 'suit': 'Clubs', 'value': 11 }
```

Before the player plays a round, the list representing the deck is created and shuffled to randomize the order of the cards. I have no graphical representation of the cards, so each time the user chooses "higher" or "lower," the program gets a card dictionary from the deck and prints the